

2.2. Minimum cost spanning trees

Optimal spanning trees have a number of applications:

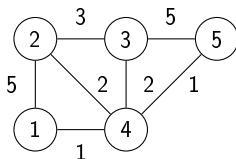
- network design (communication, electrical, ...)
- IP network protocols
- compact memory storage (DNA)
- ...

Example

Design a communication network so as to connect n cities at **minimum total cost**.

Model

Graph $G = (N, E)$ with $n = |N|$, $m = |E|$, and a cost $c_e \in \mathbb{R}$ for each $e = \{u, v\} \in E$.



Required properties:

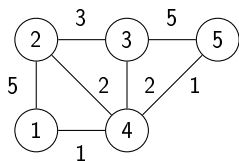
- 1 Each pair of cities must communicate $\Rightarrow$ **connected subgraph** containing **all the nodes**.
- 2 Minimum total cost $\Rightarrow$ subgraph **with no cycles**.

Problem

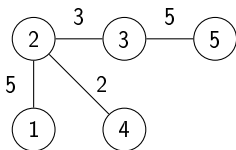
Given an undirected $G = (N, E)$ and a cost $c_e \in \mathbb{R}$ for each $e = \{u, v\} \in E$, find a spanning tree $G_T = (N, T)$ of minimum total cost.

$$\min_{T \in X} \sum_{e \in T} c_e, \text{ where } X \text{ is the set of all spanning trees of } G.$$

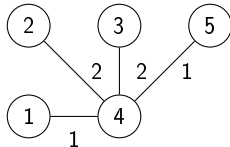
Example



G



$c(T_1) = 15$
(feasible solution)



$c(T_2) = 6$
(feasible and optimal solution)

Theorem 1 (Cayley, 1889)

A complete graph with n nodes ($n \geq 1$) has n^{n-2} spanning trees.

Example

- K_3 ($n = m = 3$) has $3^{3-2} = 3$ spanning trees.
- K_5 ($n = 5, m = 10$) has $5^{5-2} = 125$ spanning trees.

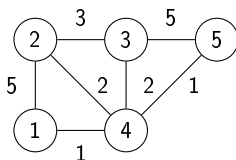
2.2.1 Prim's algorithm

Idea: Iteratively build a spanning tree.

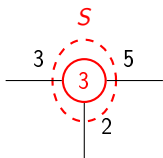
Method

- Start from initial tree (S, T) with $S = \{u\}$ and $T = \emptyset$ (u can be any node in N).
- At each step, add to the current partial tree (S, T) an edge of minimum cost among those which connect a node in S to a node in $N \setminus S$.

Example

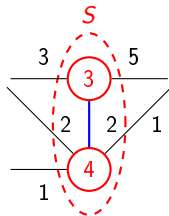


Example



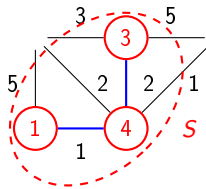
$$S = \{3\}$$

$$T = \emptyset$$



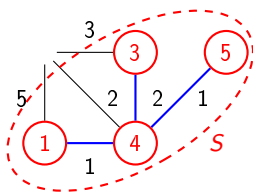
$$S = \{3, 4\}$$

$$T = \{\{3, 4\}\}$$



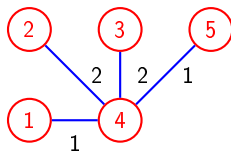
$$S = \{1, 3, 4\}$$

$$T = \{\{3, 4\}, \{1, 4\}\}$$



$$S = \{1, 3, 4, 5\}$$

$$T = \{\{3, 4\}, \{1, 4\}, \{4, 5\}\}$$



$$S = N$$

$$T = \{\{3, 4\}, \{1, 4\}, \{4, 5\}, \{2, 4\}\}$$

$$c(T) = 6$$

Prim's algorithm

- **Input:** Connected $G = (N, E)$ with edge costs
- **Output:** Subset $T \subseteq E$ of edges of G such that $G_T = (N, T)$ is a minimum cost spanning tree of G .

Algorithm 2: Prim's algorithm for the minimum cost spanning tree problem

```
1  $S \leftarrow \{u\}$ 
2  $T \leftarrow \emptyset$ 
3 while  $|T| < n - 1$  do
4    $\{u, v\} \leftarrow$  edge in  $\delta(S)$  of minimum cost /*  $u \in S$  and  $v \in N \setminus S$  */
5    $S \leftarrow S \cup \{v\}$ 
6    $T \leftarrow T \cup \{\{u, v\}\}$ 
```

If all edges are scanned at each iteration, the complexity order is: $O(nm)$.

Exactness of Prim's algorithm

Definition: An algorithm is **exact** if it provides an optimal solution for every instance, otherwise it is a **heuristic**.

Proposition 1

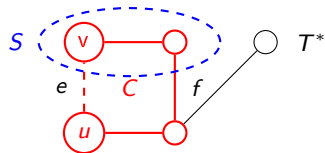
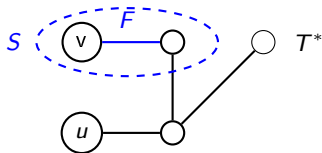
Prim's algorithm is exact.

Exact regardless of the choices of the first node and of the edge of minimum cost in $\delta(S)$.

We show that each selected edge belongs to a minimum cost spanning tree.

Property 5 (Cut property)

Let F be a partial tree (spanning nodes in $S \subseteq N$) contained in an optimal tree of G . Consider $e = \{u, v\} \in \delta(S)$ of minimum cost, then there exists a minimum cost spanning tree of G containing e .



Definition: A **greedy algorithm** constructs a feasible solution iteratively by making at each step a “locally optimal” choice, without reconsidering previous choices.

Note: Prim’s algorithm is greedy.

At each step a minimum cost edge is selected in the cut $\delta(S)$ induced by the current set of nodes S .

Note: For most discrete optimization problems, greedy-type algorithms yield a sub-optimal/poor solution.

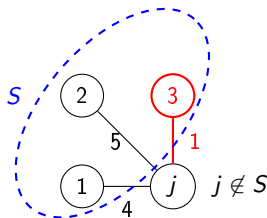
Various greedy algorithms for the minimum cost spanning tree problem:

- Prim
- Kruskal – Exercise 2.2
- ...

Implementation in $O(n^2)$

Data structure

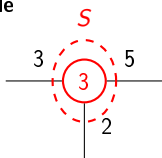
- k : number of edges selected so far
- Subset $S \subseteq N$ of nodes incident to the selected edges
- Subset $T \subseteq E$ of selected edges
- $C_j = \begin{cases} \min\{c_{ij} : i \in S\}, & j \notin S \\ +\infty, & \text{otherwise} \end{cases}$
- $closest_j = \begin{cases} \operatorname{argmin}\{c_{ij} : i \in S\}, & j \notin S \\ \text{"predecessor" of } j \text{ in the minimum spanning tree,} & j \in S \end{cases}$



$$closest_j = 3$$

$$C_j = c_{closest_j, j} = 1$$

Example

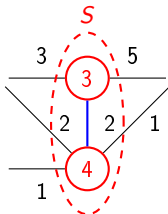


$$S = \{3\}$$

$$T = \emptyset$$

$$C = [+ \infty, 3, -, 2, 5]$$

$$\text{closest} = [3, 3, -, 3, 3]$$

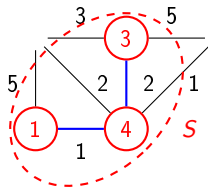


$$S = \{3, 4\}$$

$$T = \{\{3, 4\}\}$$

$$C = [1, 2, -, 2, 1]$$

$$\text{closest} = [4, 4, -, 3, 4]$$

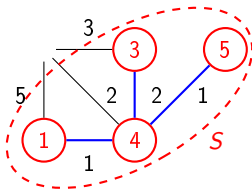


$$S = \{1, 3, 4\}$$

$$T = \{\{3, 4\}, \{1, 4\}\}$$

$$C = [1, 2, -, 2, 1]$$

$$\text{closest} = [4, 4, -, 3, 4]$$

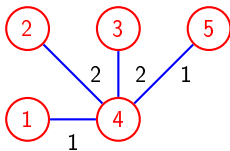


$$S = \{1, 3, 4, 5\}$$

$$T = \{\{3, 4\}, \{1, 4\}, \{4, 5\}\}$$

$$C = [1, 2, -, 2, 1]$$

$$\text{closest} = [4, 4, -, 3, 4]$$



$$S = N$$

$$T = \{\{3, 4\}, \{1, 4\}, \{4, 5\}, \{2, 4\}\}$$

$$C = [1, 2, -, 2, 1]$$

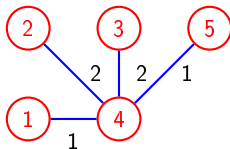
$$\text{closest} = [4, 4, -, 3, 4]$$

How to retrieve the spanning tree from *closest*?

The minimum spanning tree found by Prim's algorithm consists of the $n - 1$ edges: $\{\text{closest}_j, j\}$, with $j = 1, 2, \dots, n$ and $j \neq u$.

Example:

Since $\text{closest} = [4, 4, -, 3, 4]$, a minimum cost spanning tree consists of the edges: $\{4, 1\}$, $\{4, 2\}$, $\{3, 4\}$, $\{4, 5\}$



$O(n^2)$ version of Prim's algorithm

Algorithm 3: Prim's algorithm

```
1  $T \leftarrow \emptyset; S \leftarrow \{u\}$            /* Initialization */
2 for  $j \in N \setminus S$  do
3    $C_j \leftarrow c_{uj}$            /* If  $\{u, j\} \notin E$  then  $c_{uj} = +\infty$  */
4    $closest_j \leftarrow u$ 
5 for  $k = 1, \dots, n - 1$  do
6    $min \leftarrow +\infty$            /* Select min edge in  $\delta(S)$  */
7   for  $j = 1, \dots, n$  do
8     if  $j \notin S$  and  $C_j < min$  then
9        $min \leftarrow C_j; v \leftarrow j$ 
10   $S \leftarrow S \cup \{v\}; T \leftarrow T \cup \{\{closest_v, v\}\}$            /* Extend  $S$  and  $T$  */
11  for  $j = 1, \dots, n$  do
12    if  $j \notin S$  and  $c_{vj} < C_j$  then
13       $C_j \leftarrow c_{vj}; closest_j \leftarrow v$            /* Update  $C_j$  and  $closest_j, \forall j \notin S$  */
```

Complexity order of Prim's algorithm

```
for  $j = 2, \dots, n$  do
   $\lfloor O(1)$ 
for  $k = 1, \dots, n - 1$  do
  for  $j = 2, \dots, n$  do
     $\lfloor O(1)$ 
  for  $j = 2, \dots, n$  do
     $\lfloor O(1)$ 
```

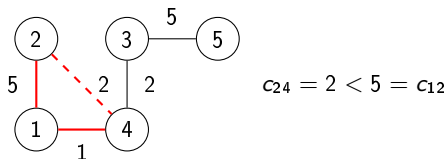
Overall complexity: $O(n^2)$

For sparse graphs, where $m \ll n(n-1)/2$, a more sophisticated data structure leads to an $O(m \log n)$ complexity.

2.2.2 Optimality condition

Definition: Given a spanning tree T , an edge $e \notin T$ is **cost decreasing** if when e is added to T it creates a cycle C with $C \subseteq T \cup \{e\}$ and $\exists f \in C \setminus \{e\}$ such that $c_e < c_f$.

Example



Because $c(T \cup \{e\} \setminus \{f\}) = c(T) + c_e - c_f$, if e is cost decreasing, then

$$c(T \cup \{e\} \setminus \{f\}) < c(T).$$

Theorem 2 (Tree optimality condition)

A tree T is of minimum total cost if and only if no cost-decreasing edge exists.

Proof's idea

⇒ If a cost-decreasing edge exists, then T is not of minimum total cost.

⇐ If no cost-decreasing edge exists, then T is of minimum total cost.

Let T^* be a minimum cost spanning tree found by Prim's algorithm.

It can be verified that, by exchanging one edge at a time, T^* can be iteratively transformed into T without modifying the total cost.

Thus, T is also optimal.

Optimality test

The optimality condition allows to **verify** whether a **spanning tree** T is **optimal**: just check that each $e \in E \setminus T$ is not a cost-decreasing edge.

2.2.3 An indirect application: optimal message passing

Given a communication network $G = (N, E)$, we want to broadcast a secret message to all the nodes so that it is not intercepted along any edge.

Problem

Let $p_{ij} \in [0, 1]$ be the probability the message is intercepted along $\{i, j\} \in E$. How to broadcast the message to all the nodes of G , so as to **minimize the probability of interception along any edge**?

Equivalent to maximize the probability of non-interception along all the edges:

$$\max_{T \in X} \prod_{\{i,j\} \in T} (1 - p_{ij}).$$

Optimal solution is a spanning tree:

- Broadcasting to all nodes $\Rightarrow$ **connected subgraph**
- To avoid redundancy and a higher probability of interception $\Rightarrow$ **acyclic subgraph**.

By applying a **monotone increasing function** (like $\log$), the objective function values change, but the optimal solutions remain unchanged.

The problem becomes:

$$\max \log \left(\prod_{\{i,j\} \in T} (1 - p_{ij}) \right) = \max \sum_{\{i,j\} \in T} \log(1 - p_{ij}).$$

which can be solved using a straightforward adaptation of any minimum cost spanning tree algorithm.